Sweet

09.2025

CLAIRE CASTAN

Holberton
SCHOOL

Partie 1

STAGE 02 : The report

09.2025

CLAIRE CASTAN

Holberton SCHOOL

task
01

Team Organization

09.2025

CLAIRE CASTAN

TEAM ORGANIZATION

09.2025

TEAM COMPOSITION

This project is carried out solo.
This means that all technical and organizational responsibilities rely on one person. Although this choice is more demanding, it provides the opportunity to work on the entire project lifecycle and gain a complete vision from design to implementation.

ROLES AND RESPONSIBILITIES

Lead Developer: responsible for backend, frontend, and database development.

Project Management: in charge of planning, progress monitoring, and meeting deadlines.

Documentation: production of required deliverables at each stage (charter, technical documentation, final report).

COLLABORATION AND ORGANIZATION STRATEGY

Communication: although there are no direct teammates, the project is developed within a pedagogical framework with regular interactions with other students and mentors.

Organization: structured planning with clear steps (ideation, documentation, development, finalization).

Shared Vision: from the very beginning, the scope has been framed to ensure a realistic, demonstrative, and scalable MVP, which will serve as the foundation for the next phases.

09.2025

task
02

Ideas explored
Idea selected

Ideas explored

THE REJECTED ONES

Idea 01

Hot Sauce Review Website

A platform where users can create an account, review hot sauces, and vote for their favorites.

Strengths:
Simple and technically feasible project.
Good demonstration of basic concepts (CRUD and authentication).

Weaknesses:
Already developed in the past.
Adds little value to the portfolio since it is redundant.

Rejected Too repetitive, lacks innovation.

Idea 02

Choice-Based Narrative Game

An interactive game where the player's choices influence the storyline.

Strengths:
Creative and original concept.
Good demonstration of logic and front-end interactivity.

Weaknesses:
Too ambitious in terms of content and development.
Impossible to finalize, even as a beta version, within the given timeframe.

Rejected Not realistic for this project

Idea 03

Children's Autonomy Application

A tool allowing parents to create daily routines for their children (e.g., brushing teeth, bedtime), which the children can validate themselves in a playful way.

Strengths:
Useful and potentially commercializable.
Strong UX potential (illustrations, gamification, routines).

Weaknesses:
Too broad in scope, requiring complex design adapted for children.
Too time-consuming for the available timeframe.

Rejected Interesting idea but better suited for a longer-term project.

IDEA SELECTED

Idea 04

Diabetese Tracking Application

A responsive web application allowing diabetic patients to track their daily health indicators (blood glucose, weight, activity, notes for medical appointments, etc.).

Strengths:
Direct personal impact (project inspired by my diabetic twin sister).
Strong utility for patients and healthcare professionals.
Demonstrates a wide range of technical skills (authentication, CRUD, data visualization, dashboard, simplified forum, etc.).
Scalable project that could be transformed into a mobile application in future years of study.

Weaknesses:
Risk of excessive scope if too many features are implemented at once.
Sensitive domain (health) → must be presented as an educational prototype, not a certified medical tool.

Strong alignment with my personal motivation, pedagogical goals, and technical feasibility.

task
03

Selected MVP

09.2025

The chosen project is a responsive web application for diabetes tracking.

It will allow users to easily record and review their daily health indicators, such as blood glucose, weight, and notes related to medical appointments. The goal is to centralize important information to simplify personal monitoring and facilitate medical consultations.

Reasons for Selection

Personal relevance:

The idea comes from a real need expressed by my diabetic twin sister, giving the project strong human and motivational value.

Utility:

The solution provides concrete benefits to diabetic patients, helping them better organize their daily monitoring.

Technical Feasibility:

In its simplified version, the features rely on skills already studied (CRUD, authentication, data visualization).

Scalability:

The project can be enriched in the future, notably during the 2nd and 3rd years, with a mobile version and advanced features (sensor integration, notifications, menstrual tracking, etc.).

Identified Challenges

Scope Management:

Avoid adding too many features at once, which could prevent delivery of a functional MVP within the timeframe.

Sensitive Domain:

Health is a delicate subject; it must be emphasized that this is an educational prototype, not a medical device.

Technical Complexity:

Some features such as graphs or the forum require good organization to be developed in a limited time.

Opportunities

Learning:

The project covers a broad spectrum of skills (authentication, database management, user interface, data visualization, managing entity relationships).

Long-term Vision:

The application will serve as a foundation for future mobile development (Flutter) and may integrate more advanced features.

Portfolio Value:

By presenting both a completed MVP and a clear roadmap, the project demonstrates my ability to combine realism with strategic vision.

task
04

Idea development Documentation

09.2025

CLAIRE CASTAN

During this first stage, several project ideas were explored. Each idea was evaluated based on its strengths, weaknesses, feasibility, and educational relevance.

Hot Sauce Review Website:

Simple and technically feasible project, but redundant with one already completed in the past. → Rejected.

Choice-Based Narrative Game:

Creative concept but unachievable within the timeframe. → Rejected.

Children's Autonomy Application:

Original and playful, but too complex for a year-end MVP. → Rejected.

Diabetes Tracking Application:

Relevant, motivating, and realistic project, covering a wide range of technical skills. → Selected.

The final choice focused on the diabetes tracking application because:

- it addresses a real and concrete need,
- it is technically feasible within the timeframe,
- it allows me to put into practice the majority of the skills acquired this year,
- it offers an evolutionary vision that can be pursued and enriched in the coming years.

This selection process made it possible to frame a clear and achievable MVP while laying the foundations for a future roadmap.

The work carried out in this first stage ensures that the next phases (project charter, technical documentation, and development) can rely on a solid, realistic, and shared vision.

Partie2

STAGE 02 : The report

09.2025

CLAIRE CASTAN

Holberton SCHOOL

09.2025

task
01

Main objectives
SMART objectives

USER STORIE

USER STORIES AND WIREFRAMES

The Sweet application aims to centralize essential information related to diabetes monitoring in order to make patients' daily disease management easier.

It seeks to provide a clear and personalized overview of health status, while enabling patients to prepare for medical appointments with reliable data, and benefit from the support of a peer community.

MAIN OBJECTIVES

1

Centralization and visualization of health data

Provide patients with the ability to record and view their data (blood glucose, weight, physical activity, meals, medication intake + stock, mood).

Offer a clear visual representation (day, week, month) to track progress and better understand trends.

2

Improved medical follow-up

Help patients prepare for their medical appointments with precise, centralized, and historical data.

Provide healthcare professionals with concrete and reliable information (rather than estimates). This facilitates a more accurate diagnosis and adjustment of treatment protocols based on the patient's lifestyle.

3

Organization and personalized monitoring

Provide a calendar to consult or edit historical data and automatically generate weekly and monthly reports.

Allow patients to add notes during medical appointments to improve follow-up and anticipate their needs.

4

Creation of a support community

Provide a forum for patients to share their experiences, difficulties, recipes, or tips.

Combat patient isolation by promoting sharing and solidarity within a community of peers.

SMART OBJECTIVES

1

Centralization and visualization of health data

Enable patients to record and view their daily data (blood glucose, weight, physical activity, meals, medications, and mood) through a functional daily log, operational by week 8 of the project.

2

Improved medical follow-up (daily dashboard)

Provide a clear and ergonomic dashboard allowing patients to view their daily data in real time, delivered and testable by week 8.

3

Organization and personalized monitoring (calendar and reports)

Develop an interactive calendar allowing consultation or modification of historical data and automatic generation of weekly and monthly reports, available and tested before week 9.

4

Creation of a support community (forum)

Deploy a basic forum (topic creation and comments), with at least 80% of functionalities operational no later than week 10, to encourage patient exchanges.

09.2025

task
02

In-scope
Out-of-scope

Project Scope

CLAIRE CASTAN

PROJECT SCOPE

The Sweet application focuses on simple, reliable, and community-based diabetes monitoring. The project scope defines the features and objectives included in the first version (MVP), as well as those voluntarily excluded to ensure feasibility within the given timeline.

In-scope

MANAGEMENT OF ESSENTIAL HEALTH DATA

Manual entry and consultation of blood glucose data.
Addition and monitoring of weight and physical activity.
Generation of simple statistical reports (average, minimum, maximum).

DASHBOARD AND VISUALIZATION

Synthetic view of data by day/week/month.
Simplified graphs to illustrate trends.

PERSONAL ORGANIZATION

Simplified calendar for medical appointments and data consultation.

Ability to add associated notes.

COMMUNICATION AND COMMUNITY

Basic forum (topic creation, comments).

Articles/news section, managed by the administrator.

MEDICAL CONTACTS MANAGEMENT

Adding essential information about healthcare professionals (name, phone number, note-taking during appointments).

Out-of-scope

Synchronization with connected medical devices (blood glucose sensors, smartwatches, etc.).

Automated medical analysis or advanced medical assistance (automatic diagnosis, medical recommendations).

Advanced social features (private messaging, complex discussion groups).

Multilingual interface (only available in French for the MVP).

Native mobile application (the MVP will be a responsive web app).

09.2025

task
03

End-users
Project team
External stakeholders

Project Stakeholders

CLAIRE CASTAN

END-USERS

DIABETIC PATIENTS

Main actors and sole direct users of the application.

Need:
centralize health data, track progress, and prepare medical appointments.

Full control:
patients decide when and how to show their data to a doctor, with no automatic sharing.

INDIRECT USAGE (VIA PATIENTS)

Healthcare professionals (doctors, nurses, diabetologists)

No direct access to the application.

Patients may decide to present their dashboard or reports during consultations.

Their role is therefore indirect:
using information provided by the patient to adjust treatments and recommendations.

EXCLUSIONS

Relatives and caregivers

No access to the application.

Health data is strictly private, and only patients decide on any potential sharing.

PROJECT TEAM

DEVELOPER & PROJECT MANAGER:
CLAIRE CASTAN

Responsible for backend, frontend, and database development.

Project coordination, timeline management, and deliverable organization.

Supervision of documentation and final presentation.

UI/UX DESIGNER:
CLAIRE CASTAN

Design of the interface and user experience.

Ensuring clear, intuitive navigation adapted to the target audience.

TESTERS / QUALITY:
CLAIRE CASTAN, DIABETIC PATIENT, OTHERS

Verification of developed functionalities.

Bug identification and suggestions for improvements.

Feedback on user experience to refine the application.

EXTERNAL STAKEHOLDERS

EDUCATIONAL SUPERVISORS:
accompany and evaluate each stage of the project.

EVALUATORS/JURY:
assess technical, organizational, and documentary quality during the final presentation.

END-USERS (DIABETIC PATIENTS):
considered stakeholders since the application is designed to meet their needs.

09.2025

task
04

Technical risks
Organizational risks
User-related risks

Project risks

CLAIRE CASTAN

TECHNICAL RISKS

CHOICE OF TECHNOLOGIES TOO COMPLEX OR UNSUITABLE

Risk: wasted time mastering the stack, project delays.

Mitigation: prioritize known and suitable technologies for the MVP (simple stack, familiar frameworks).

SECURITY AND CONFIDENTIALITY OF HEALTH DATA

Risk: exposure of sensitive data if authentication or storage is poorly managed.

Mitigation: implementation of reliable basic authentication, adherence to best storage practices (even with fictitious data for the project).

LACK OF TECHNICAL TESTING

Risk: bugs during demonstration, loss of credibility.

Mitigation: testing phase planned at the end of the project, testers/ QA involved early in development.

ORGANIZATIONAL RISKS

DELAYS IN SCHEDULING

Risk: some secondary features (forum, articles) not completed on time.

Mitigation: focus on absolute priorities (authentication + health dashboard) before extras.

INCOMPLETE OR SCATTERED DOCUMENTATION

Risk: deliverable not compliant with school expectations.

Mitigation: progressive documentation writing, centralization in a shared folder.

STRESS AND WORKLOAD FOR THE ORAL DEFENSE

Risk: lack of clarity during final presentation.

Mitigation: early preparation of defense (slides + pitch), rehearsals before the deadline.

USER-RELATED RISKS

LIMITED ADOPTION OF THE CONCEPT BY PATIENTS

Risk: perception of the tool as too technical or guilt-inducing.

Mitigation: simple interface, patient-centered, no third-party access (doctors/relatives) to ensure confidentiality.

MISINTERPRETATION OF DATA BY USERS

Risk: patients relying solely on the app without medical advice.

Mitigation: clearly remind that the app is a monitoring tool, not a certified medical device.

09.2025

task
05

Project technologies

STAGE 02 : The report

CLAIRE CASTAN

Holberton
SCHOOL

PROJECT TECHNOLOGIES

09.2025

BACKEND – FLASK (PYTHON)

Choice: use of Flask as backend framework.

Justifications:

- Lightweight and modular framework, perfectly suited for an MVP.
- Rich ecosystem (Flask-SQLAlchemy, Flask-Login, Flask-JWT, Flask-Migrate) allowing only necessary additions.
- Easy setup of a REST API to expose data to the frontend.
- Built-in test client, facilitating unit test implementation.
- Performance sufficient for an MVP with limited traffic.
- Prior experience with Flask and Python (leveraging acquired skills).

DATABASE – MYSQL

Choice: relational database MySQL with SQLAlchemy as ORM.

Justifications:

- Stable database, widely used in enterprises.
- Perfectly suited for relational needs (one patient → multiple glucose entries, weights, activities, etc.).
- Standardized SQL language, skills easily transferable to PostgreSQL/MariaDB.
- Native relational constraints (primary and foreign keys) ensuring data consistency.
- Smooth integration with Flask via SQLAlchemy.
- Prior experience with MySQL.

AUTHENTICATION AND SECURITY

Choice: authentication management with JWT (JSON Web Tokens) or Flask-Login.

Justifications:

- Professional best practice, essential when handling personal data.
- Stronger security compared to basic sessions.
- JWT is a market standard, used in many modern applications.
- Scalable: if the app is deployed to the cloud, JWT adapts better than traditional sessions.

Additional measure:

Display of a clear disclaimer in the interface:
"Sweet is a personal monitoring tool. It does not in any way replace the advice of a qualified healthcare professional."

FRONTEND – REACT

Choice: use of React to build the user interface.

Justifications:

- React is the most in-demand frontend library on the market, strategically aligned with recruiter expectations.
- Reusable components facilitate maintenance (e.g., data entry forms).
- Virtual DOM offers better performance for dynamic dashboards (glucose data day/week/month).
- Mature ecosystem: easy integration of libraries like React Router (navigation) or Chart.js/Recharts (visualization).
- Clear separation between backend and frontend → modern, clean, and scalable architecture.
- Prior experience with React, reinforced and consolidated in this project.

DATA VISUALIZATION – CHART.JS INTEGRATED WITH REACT

Choice: integration of Chart.js to graphically represent glucose, weight, and activity data.

Justifications:

- Lightweight yet powerful library, perfectly suited for an MVP.
- Simplifies patient understanding of data (day/week/month graphs).
- Strong visual impact during the demonstration.
- Customization options (colored thresholds for abnormal values).

PROJECT TECHNOLOGIES

09.2025

CROSS-CUTTING BENEFITS OF THIS STACK

Complementarity:
Flask (backend API) + React (frontend) + MySQL (DB) is a modern architecture, widely used in enterprises.

Realism:
Stack simple enough for an MVP but robust enough for a defense.

Scalability:
Possibility to later add cloud hosting, external APIs, or a mobile app in React Native.

Attractiveness:
strategic choices that highlight skills for a future full-stack developer position oriented towards frontend.

MANAGEMENT AND SUPPORT TOOLS

GitHub:
project versioning, professional standard.

Figma:
UI/UX prototyping before implementation, ensuring a clear and user-adapted interface.

09.2025

task
06

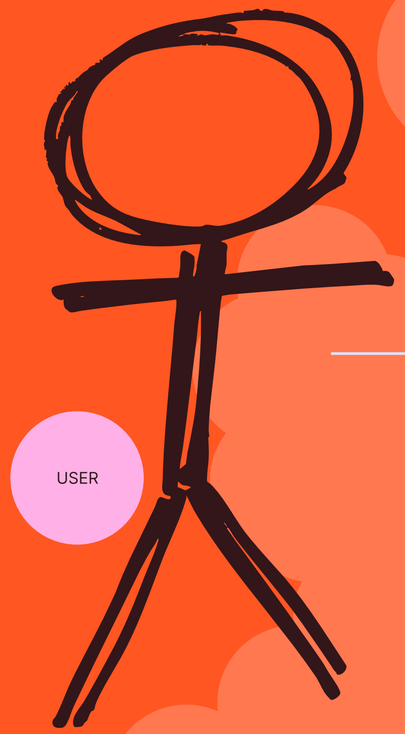
High-level UML diagram

STAGE 02 : The report

CLAIRE CASTAN

Holberton
SCHOOL

HIGH-LEVEL UML DIAGRAM



Sweet

Authentification
- Signup
- Login
- Logout

Daily log

Health data
- Glucose
- Weight
- Physical activity

Context
- Meals
- Medi-cation
- Mood

Write a consultation report
- Add a medical appointment
- Link a healthcare professional
- Medical follow-up
- View consultation history

Community
- Create a forum topic
- Comment on a topic
- Read article/news

Authentication & Account

- Create an account (sign up).
- Log in.
- Log out.

Daily Monitoring (daily form)

- Fill out the daily form with the following items:
  - Blond glucose.
  - Weight.
  - Physical activity.
  - Meals consumed (without calorie calculation, simple description).
  - Medication intake (monitoring + stock management).
  - Mood (on a scale or free comment).
- Modify the daily form several times a day (additions, corrections, adjustments).
- Automatic saving to the database when the day changes.

Consultation & modification of past data

- Open the calendar to select a specific day.
- Consult the form for that day (blood glucose, weight, activity, meals, mood, medications).
- Edit a previous day (e.g., add a missed activity or meal).
- Access weekly and monthly reports directly from the calendar (average, minimum, maximum)..

Organization & medical follow-up

- Add a medical appointment to the calendar.
- Link a healthcare professional to an appointment (example: “Dr Go – diabetologist”).
- Write a personal consultation report (text area for advice, adjustments, or tips given by the doctor).
- Consult the history of medical appointments and their associated notes.

Community & Information

- Create a forum topic.
- Comment on an existing topic.
- Read articles/news published by the administrator.

09.2025

task
07

Conclusion and Final Delivery

CLAIRE CASTAN

Holberton
SCHOOL

HIGH-LEVEL PLAN

STAGE 01

Idea Development → project selection and idea framing.

STAGE 02

Project Charter → definition of objectives, scope, risks, stakeholders, UML. (weeks 3–4)

STAGE 03

Technical Documentation → DB schema, API endpoints, front/back architecture. (weeks 5–6)

STAGE 04

MVP Development (weeks 7–10)

Week 7–8: Development of the daily form + daily dashboard (Objectives 1 and 2).

Week 9: Development of the interactive calendar + reports (Objective 3).

Week 10: Development of the basic forum (Objective 4).

STAGE 05

Project Closure → defense and final documentation. (weeks 11–12)

The Sweet project was designed as a patient-centered pathology monitoring application, with the aim of making daily diabetes management simpler, clearer, and more effective.
The technical choices (Flask, React, MySQL) reflect the desire to combine medical data security, clean architecture, and smooth user experience.

CONCLUSION AND FINAL DELIVERY

The Sweet project was designed as a patient-centered pathology monitoring application, with the aim of making daily diabetes management simpler, clearer, and more effective.

The technical choices (Flask, React, MySQL) reflect the desire to combine medical data security, clean architecture, and smooth user experience.

The system is structured around essential features:

Daily monitoring
(daily form: blood glucose, weight, activity, meals, medications, mood).

Consultation and reports
(via a central calendar).

Medical organization
(appointments, healthcare professionals, consultation notes).

Community
(forum, articles).

Special attention has been given to confidentiality: the patient remains the sole owner of their data, choosing whether or not to share it with their doctor.

The mobile-first design ensures an interface adapted to real patient use and already prepares the transition to a native mobile application. Thanks to this modular architecture (API REST + React frontend), the project can evolve without major redesign into Flutter or React Native, offering a complete mobile experience.

PLANNED DELIVERABLES

Project Charter document (this report).

High-Level UML diagram (Use Cases).

Documentation of technical and functional choices.

FUTURE VISION

The Sweet project aims to go beyond the academic framework to become a truly useful application for diabetic patients in their daily lives.
In the long term, the goal is to provide a solution:

- Secure: respecting confidentiality standards and protecting sensitive data.
- Accessible: mobile-first, intuitive, and adapted to all patient profiles.
- Scalable: able to progressively incorporate new features (integration of connected devices, enhanced medical monitoring, educational tools).

The ultimate goal is for Sweet to be genuinely used by diabetic patients as a centralized monitoring tool, helping them gain clarity and simplicity in managing their condition, and thereby making their daily life easier.

Partie 03

STAGE 03 : The report

09.2025

CLAIRE CASTAN

Holberton SCHOOL

09.2025

task
00

User stories
Wireframe

User stories and wireframes

CLAIRE CASTAN

USER STORIES

1

Must Have

As a user, I want to enter my blood glucose measurements in order to track their evolution and quickly detect imbalances.

As a user, I want to track my weight evolution to measure the impact of my diet and physical activity on my health.

As a user, I want to record my physical activities to keep a trace and follow their evolution.

As a user, I want to record my meals to identify which foods influence my blood sugar and better understand their effects.

As a user, I want to record my medication intakes to avoid forgetting and respect my treatment.

As a user, I want to track my emotional state and stress level to understand their possible influence on blood sugar variations.

As a user, I want to access a dashboard aggregating my data (day/week/month) to get a quick and clear overview.

As a user, I want to add notes associated with my medical appointments to keep track of the advice and adjustments given by my doctor.

As a user, I want to consult weekly and monthly reports directly from the calendar in order to have a regular vision of my health.

2

Should Have

As a user, I want to consult the history of my blood glucose levels in order to better prepare my appointments and have reliable information.

As a user, I want to visualize my weight trends (stability, loss, gain) in order to better understand my overall balance.

As a user, I want to consult the history of my physical activities in order to understand their impact on my blood sugar and choose those that help me regulate it best.

As a user, I want to track the stock of my medications to anticipate renewals and not run out.

As a user, I want to automatically generate statistical summaries (average, minimum, maximum) to have a
As a user, I want to visualize my health data as charts to better understand their evolution.

As a user, I want to consult and modify the history of my data via a calendar in order to correct or complete my past information.

As a user, I want to add a medical contact (name, phone, notes) in order to centralize my useful information.

As a user, I want to consult and modify my list of medical contacts to always have up-to-date information.

3

Could Have

As a user, I want to create a topic on the forum in order to share a question, an experience or a tip with the community.

As a user, I want to comment on a forum topic in order to interact with other users.

As a user, I want to exchange with other diabetics through a forum in order to break isolation and share my experience.

As a user, I want to consult articles or news published by the administrator to stay informed about topics related to diabetes and health.

4

Won't Have (MVP)

As a user, I want to automatically sync my blood sugar data with a connected sensor so I don't have to enter it manually.

As a user, I want to receive automatic medical recommendations to adapt my treatment without consulting my doctor.

As a user, I want to send private messages to other community members in order to exchange more personally.

As a user, I want to use the app in several languages to access it in my native language.

As a user, I want to install Sweet as a native mobile app to access it directly from my smartphone without going through the browser.

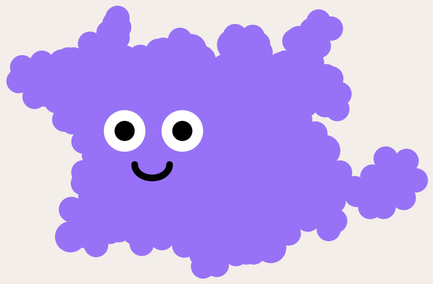
WIREFRAMES

Sweet

Se connecter

Inscription

WIREFRAMES



Sweet

Nom d’utilisateur / Adresse mail

Mot de passe

Se souvenir de moi

Mot de passe oublié ?

Se connecter

WIREFRAMES

Avec quels identifiants veux-tu te connecter ?

Adresse mail

Mot de passe

Requis
x Minimum 8 caractères
x Une majuscule, minuscules, et un caractère spécial

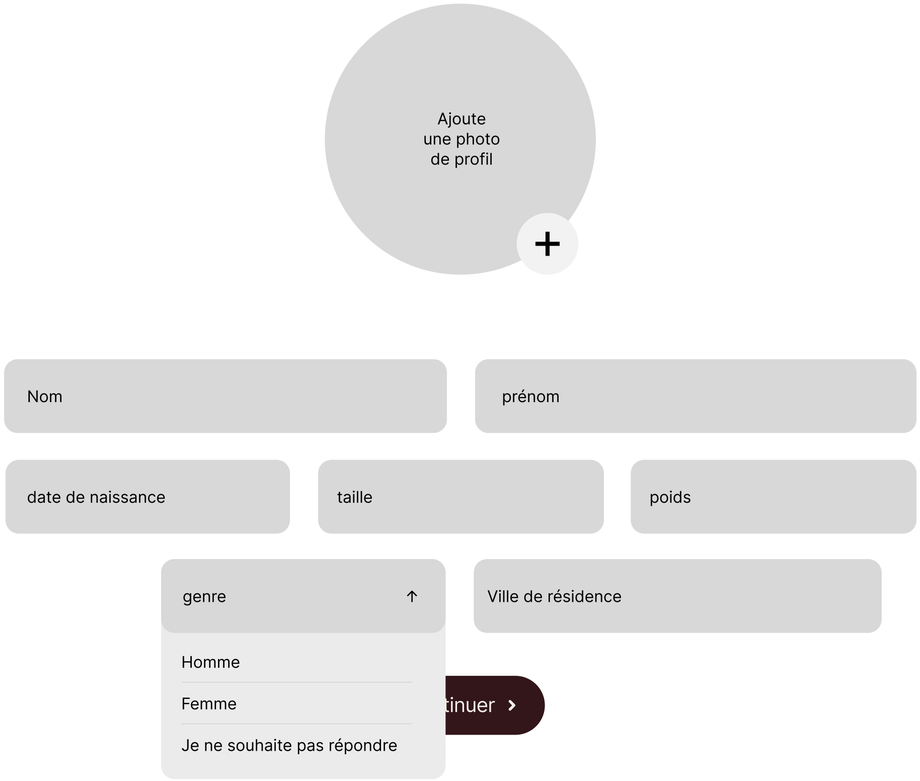
Confirmez le mot de passe

Nom d’utilisateur

En continuant, vous acceptez les Conditions d’utilisation de Sweet et vous confirmez avoir lu la Politique de confidentialité de Sweet.

Continuer ›

WIREFRAMES


Ajoute
une photo
de profil
Nom
prénom
date de naissance
taille
poids
genre
Ville de résidence
Homme
Femme
Je ne souhaite pas répondre
tinuer

WIREFRAMES

Qui sont tes professionnels de santé ?

Compléter plus tard ›

Médecin généraliste	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	
		notes-janvier.txt	
Diététicien	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	
Infirmier	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	−
Diabétologue	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	−
Psychologue	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	−
Pédicure podologue	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	−
Endoctrinologue	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	−
Ophtalmologue	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	−
Cardiologue	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	−
Néphrologue	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	−
Néphrologue	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	−

Spécialité (Requis)	Nom (Requis)	N° de téléphone	Ajouter des documents (.jpg, .txt, .pdf) +	Valider

Rajouter un professionnel de santé +

‹ Retour Continuer ›

WIREFRAMES

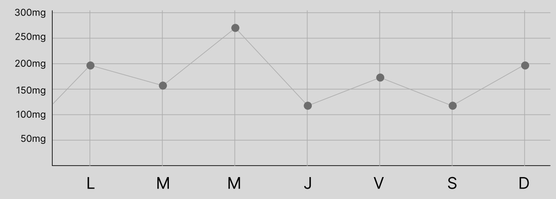
Bonjour XXX !

Aujourd’hui nous somme le :
date du jour

pp

Glycémie

Cette semaine vous avez en moyenne fait : 150mg/l



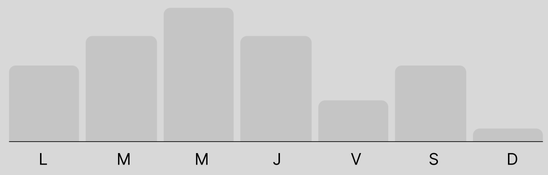
Votre moyenne :

30 jours | 90 jours

150mg/l

Activité physique

Cette semaine vous avez en moyenne fait : 45min/j



Votre moyenne :

30 jours | 90 jours

150min/j

Humeur

Mon humeur aujourd’hui :

Repas

Aujourd’hui, j’ai mangé :

Petit-déjeuner :

Oeufs au plat, bacon, pain complet

Déjeuner :

Lorem Ipsum

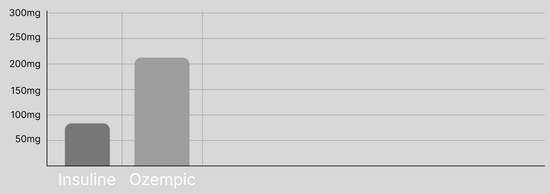
Dîner :

Lorem Ipsum

En-cas :

Lorem Ipsum

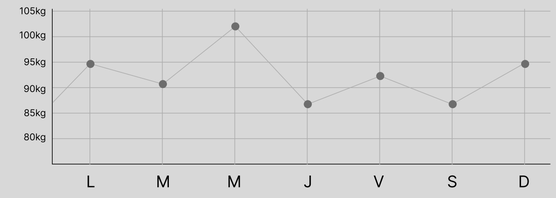
Médication



Lorem Ipsum

Poids

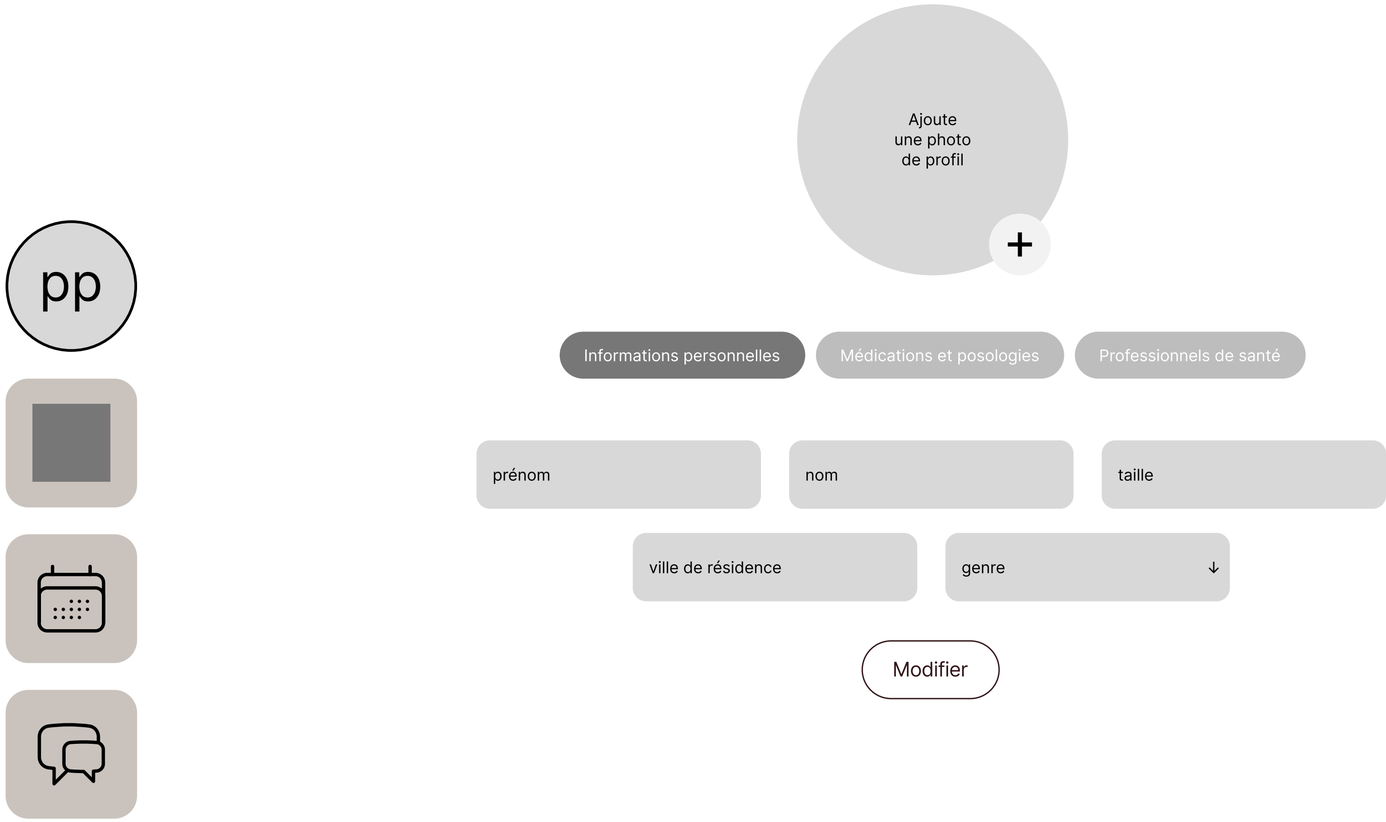
7 jours | 30 jours | 90 jours | 6 mois



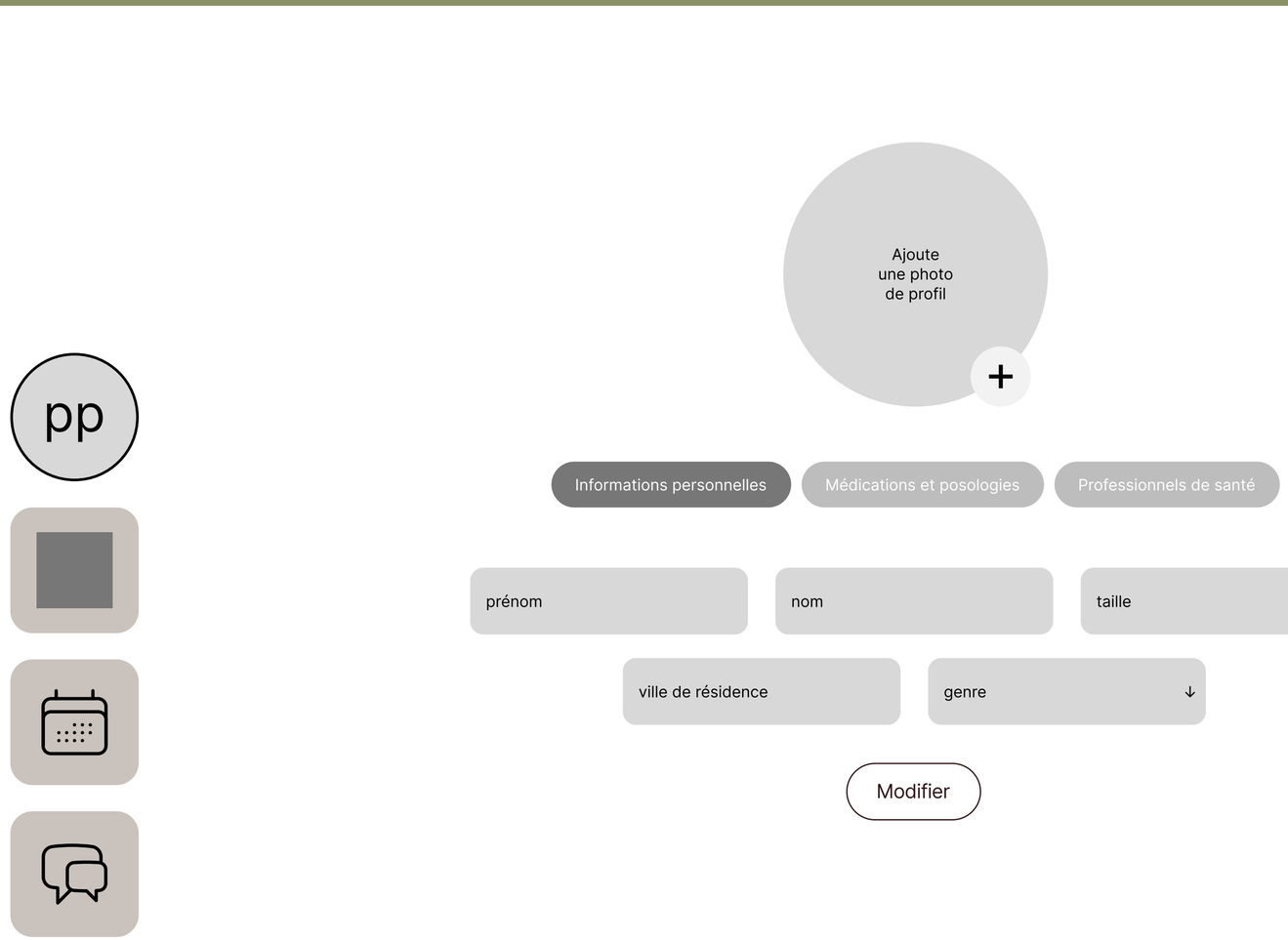
Actualités

Découvrir
toutes nos acutalités

WIREFRAMES


Ajoute
une photo
de profil
+
pp
Informations personnelles
Médications et posologies
Professionnels de santé
prénom
nom
taille
ville de résidence
genre
Modifier

WIREFRAMES


pp
Ajoute
une photo
de profil
+
Informations personnelles
Médications et posologies
Professionnels de santé
prénom
nom
taille
ville de résidence
genre
Modifier

09.2025

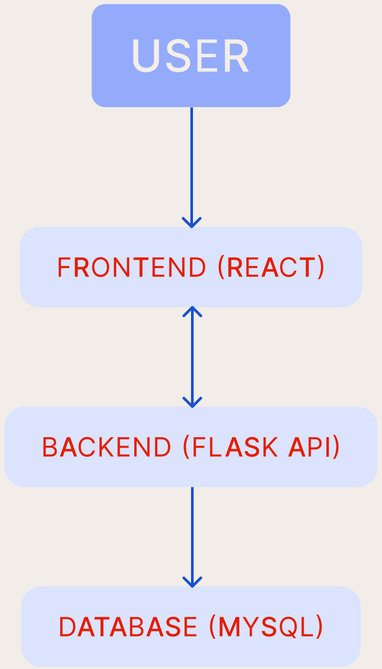
task
01

ARCHITECTURE SYSTEM
FRONTEND
BACKEND
DATABASE
SECURITY

System Architecture

CLAIRE CASTAN

SYSTEM ARCHITECTURE



FRONTEND

The frontend is developed with **React.**

It manages the user interface and navigation between different pages (login, dashboard, forum, articles, contacts, calendar).

Global state is centralized via React Context, allowing essential information to be shared (authenticated user, health data, display preferences).

The graphical interface is built with modular SASS, ensuring a lightweight layout aligned with eco-design principles, with simple, non-intrusive animations.

Data visualization is powered by Chart.js, providing clear charts (glycemia, weight, activity).

Authentication uses JWT tokens stored in secure httpOnly cookies (see Security section).

BACKEND

The backend is built with Flask.

Its architecture is organized into Blueprints, one module per domain (auth, data, forum, articles, contacts).

It exposes a versioned RESTful API (/api/v1/) for creating, updating, and retrieving data.

Endpoints use GET, POST, and PUT. An exception is made for articles, managed by the administrator, which support a complete CRUD cycle (GET, POST, PUT, DELETE).

Authentication is managed with JWT:
- an access token valid for 24h,
- and a refresh token valid for 48h if the user selects “Remember me.”

Two roles are defined:
- User: access to their personal data, the forum, and article reading.
- Administrator: management of articles and future moderation tools.

Data validation is handled directly in the model classes (constructors and setters), using regex for sensitive fields (email, password, phone) and simple checks for numerical fields (glycemia, weight, etc.).

Errors are returned in a uniform JSON format to simplify debugging and frontend display.

DATABASE

The chosen database is MySQL, configured in utf8mb4 to ensure compatibility with special characters and emojis. Access is managed through SQLAlchemy (ORM), with Flask-Migrate handling migrations.

Main tables cover the functional needs of the application:

- User: account and role management (user or administrator).
- Health data: glycemia, weight, meals, activities, mood, medication.
- Organization: medical appointments and professional contacts.
- Forum: posts and comments.
- Articles: content created and managed by the administrator.

Relationships follow standard logic:

- one user can have multiple health data entries, appointments, contacts, or forum posts,
- one post can have multiple comments,
- articles are linked only to the administrator.

Indexes are applied to foreign keys and frequently used fields (e.g., user_id and date in health data) to optimize queries while adhering to eco-design principles.

SECURITY

The Sweet application implements several security measures appropriate for an MVP, while remaining extensible for real-world use.

Authentication and roles
- Authentication with JWT stored in httpOnly cookies.
- access_token valid 24h, refresh_token valid 48h if “Remember me” is enabled.
- Distinct roles:
  User → access restricted to personal data and forum features.
  Admin → create and manage articles through dedicated routes (/api/v1/admin/*).

Passwords
- Secure storage with bcrypt.
- Complexity policy: at least 8 characters, including 1 digit and 1 letter.
- Regex validation and blacklist of overly simple passwords.
- “Forgot password” feature planned, with reset link sent by email.

Database security
- utf8mb4 encoding for full compatibility with special characters and emojis.
- No encryption of health data in the MVP, but strict access control via user_id and JWT.
- Protection against injection via SQLAlchemy ORM, and XSS filtering on text fields (forum, notes).

Input validation
- Regex for sensitive fields (email, password, phone).
- Boundary checks for numeric data (glycemia, weight, activity).
- Strict enforcement of business rules (a user can only access their own data, activities must belong to valid types, etc.).

Error handling and logs
- Generic error messages on the frontend, uniform JSON format for all backend errors.
- Backend → logging (INFO/WARNING/ERROR), never storing passwords or tokens.
- Frontend → errors forwarded to the backend for centralized logging.

Data transport
- HTTPS mandatory in production (Let’s Encrypt).
- Secure cookies (HttpOnly + Secure + SameSite=Lax).
- Strict CORS policy: localhost allowed in dev, final domain in production.
  Request size limited (~2 MB).
  Possibility to add security headers (CSP, X-Frame-Options).
-

Define components, classes, and Database design

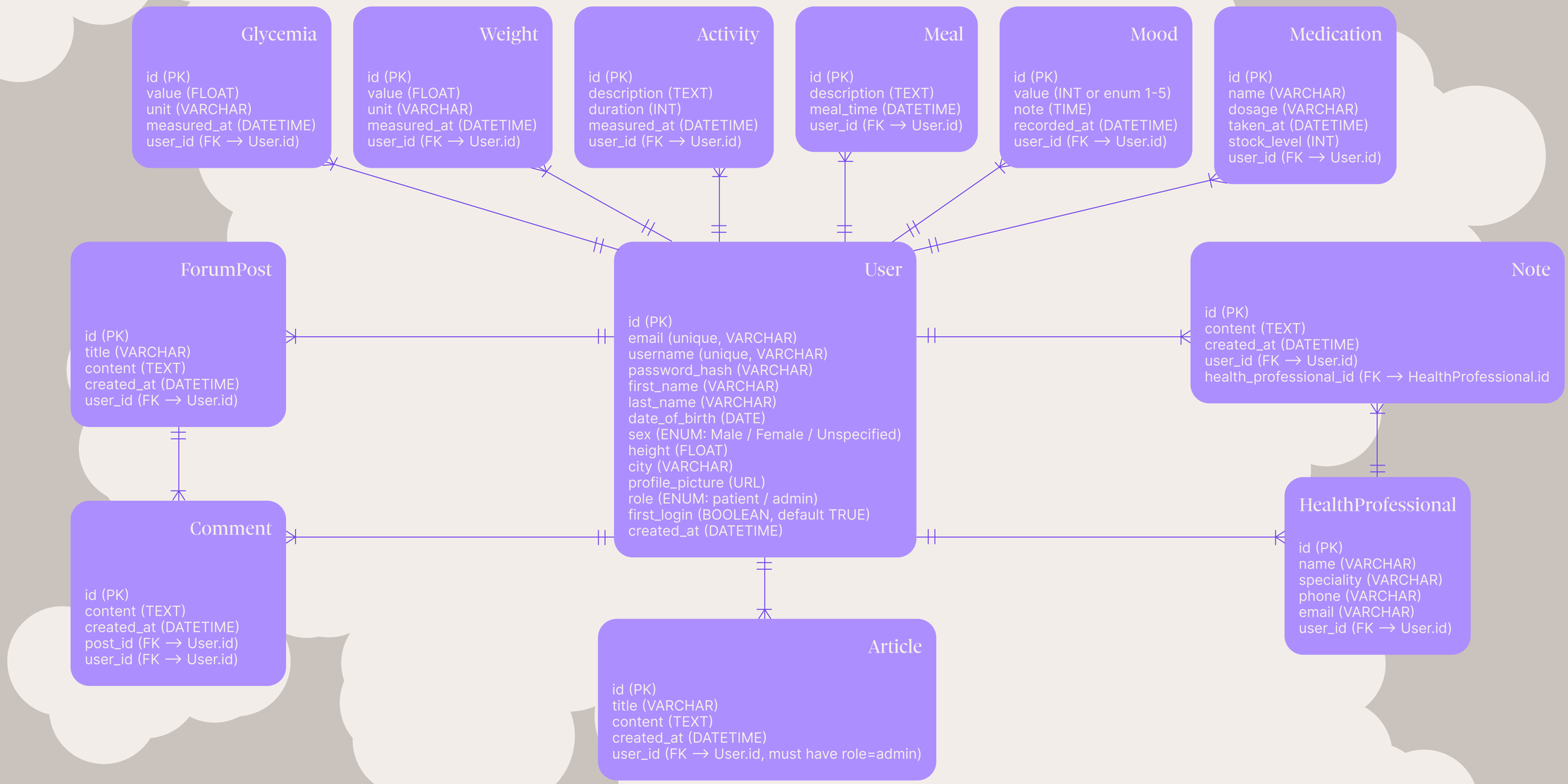
task
02

DATABASE
BACKEND ORGANIZATION
FRONTEND ORGANIZATION

09.2025

CLAIRE CASTAN

DATABASE


Glycemia
id (PK)
value (FLOAT)
unit (VARCHAR)
measured_at (DATETIME)
user_id (FK → User.id)
Weight
id (PK)
value (FLOAT)
unit (VARCHAR)
measured_at (DATETIME)
user_id (FK → User.id)
Activity
id (PK)
description (TEXT)
duration (INT)
measured_at (DATETIME)
user_id (FK → User.id)
Meal
id (PK)
description (TEXT)
meal_time (DATETIME)
user_id (FK → User.id)
Mood
id (PK)
value (INT or enum 1-5)
note (TIME)
recorded_at (DATETIME)
user_id (FK → User.id)
Medication
id (PK)
name (VARCHAR)
dosage (VARCHAR)
taken_at (DATETIME)
stock_level (INT)
user_id (FK → User.id)
ForumPost
id (PK)
title (VARCHAR)
content (TEXT)
created_at (DATETIME)
user_id (FK → User.id)
User
id (PK)
email (unique, VARCHAR)
username (unique, VARCHAR)
password_hash (VARCHAR)
first_name (VARCHAR)
last_name (VARCHAR)
date_of_birth (DATE)
sex (ENUM: Male / Female / Unspecified)
height (FLOAT)
city (VARCHAR)
profile_picture (URL)
role (ENUM: patient / admin)
first_login (BOOLEAN, default TRUE)
created_at (DATETIME)
Note
id (PK)
content (TEXT)
created_at (DATETIME)
user_id (FK → User.id)
health_professional_id (FK → HealthProfessional.id
Comment
id (PK)
content (TEXT)
created_at (DATETIME)
post_id (FK → User.id)
user_id (FK → User.id)
HealthProfessional
id (PK)
name (VARCHAR)
speciality (VARCHAR)
phone (VARCHAR)
email (VARCHAR)
user_id (FK → User.id)
Article
id (PK)
title (VARCHAR)
content (TEXT)
created_at (DATETIME)
user_id (FK → User.id, must have role=admin)

SWEET DATABASE DESIGN

The database design for Sweet clearly separates user data, health tracking, community features, and health professionals. The model follows best practices: normalized tables, one-to-many relationships, and special attention to security and confidentiality of medical data. Crow's foot notation is used to represent cardinalities.

User

The User table is central. It stores authentication information (email, username, password), profile data (last name, first name, birthdate, gender, height, city, profile picture), as well as technical fields (role, first_login, creation date).

Each user can be either a patient or an administrator (role field).
⚠ For security reasons, the admin role is never chosen by the user.

Users created through registration always have the patient role.

Administrator accounts are defined only directly in the database (by a system admin or via a secure script).

Administrators are the only ones allowed to publish articles.

A boolean field first_login displays the tutorial only on the first login.

Health data

Health data is historized in separate tables for each metric:

Glycemia

Weight

Activity (physical activity)

Meal

Medication (drug, dosage, stock, intake date)

Mood (scale 1–5 with optional note)

Each table records a timestamp (measured_at or recorded_at for mood) and is linked to User through a one-to-many relationship.

Articles

The Article table is used to publish educational or informational content.

Only users with role = admin can publish articles.

Each article is linked to a record in the User table via user_id.

Health professionals and notes

The HealthProfessional table allows each user to store their own health professionals (name, specialty, contact details).

⚠ For confidentiality, each professional is linked only to one user (user_id). This ensures no sharing or cross-referencing of medical information between patients.

The Note table stores remarks, advice, or consultation summaries.
Each note is linked to:
a User (author of the note),
a HealthProfessional (the professional concerned).

Conclusion

This design ensures:

Confidentiality: each patient has their own private dataset, including their health professionals.

Traceability: all health data is historized with timestamps.

Scalability: new metrics (e.g., sleep) or new community features can be added without redesign.

The ERD highlights the central role of User, around which health data, professionals, notes, and community features are organized.

Community features

The ForumPost and Comment tables manage social interactions:

A user can create multiple posts.
Each post can receive multiple comments.
Each comment is linked to its author.

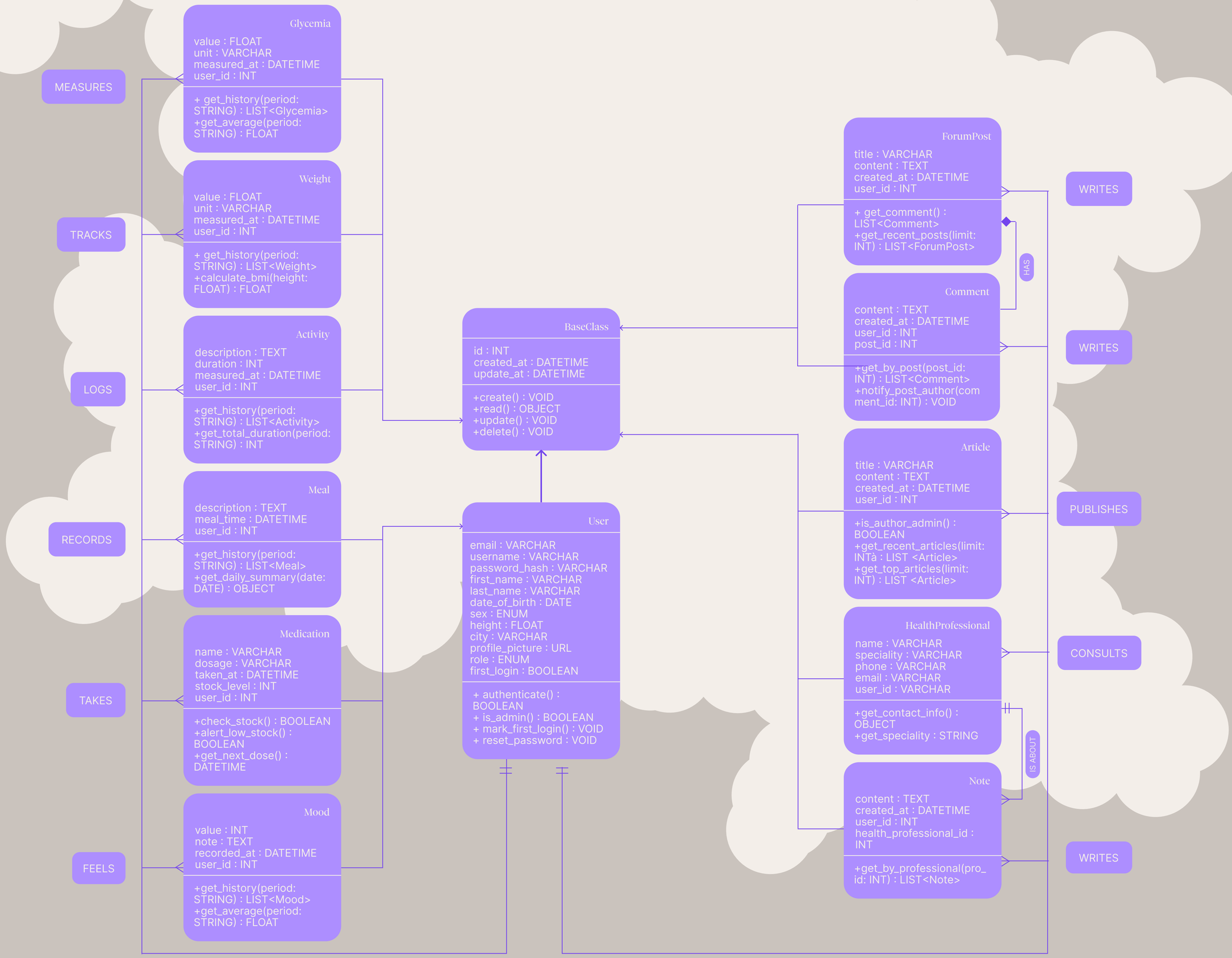
This structure guarantees a clear, scalable forum, decoupled from medical data.

Directory structure

```
database/
├── tables/
│   ├── user.sql
│   ├── glycemia.sql
│   ├── weight.sql
│   ├── activity.sql
│   ├── meal.sql
│   ├── medication.sql
│   ├── mood.sql
│   ├── health_professional.sql
│   ├── note.sql
│   ├── forum_post.sql
│   ├── comment.sql
│   └── article.sql
└── migrations/
    └── init.sql
```


CLAIRE CASTAN

BACKEND ORGANIZATION



DEFINE COMPONENTS, CLASSES, AND DATABASE DESIGN

09.2025

The Sweet class diagram illustrates the logical structure of the backend and the relationships between the application entities.

General structure

All entities inherit from an abstract BaseClass that centralizes CRUD methods (create(), read(), update(), delete()), as well as common attributes (id, created_at, updated_at).
This approach ensures code consistency and avoids duplication of core functionality.

User and health data

The User class is at the heart of the system:
It manages authentication, roles (patient or admin), the first login tutorial, and password reset.
It is linked one-to-many with Glycemia, Weight, Activity, Meal, Medication, and Mood.

Each relationship reflects a user action:
- a user measures glycemia,
- tracks weight,
- logs activities,
- records meals,
- takes medication,
- feels and notes mood.

Health professionals and notes

A User can consult one or more Health Professional. The user can then write a Note about the consultation, linked both to the professional (is about) and to the user.

Community forum

The community area is represented by:
ForumPost, written by a user,
Comment, written by a user but attached to a post.

This relationship is a composition: a comment cannot exist without its associated post.

Articles

Article entities are published only by users with the admin role.

All users can read them, but only administrators are authors.

Conclusion

This class diagram clearly shows inheritance, relationships, and business responsibilities:
inheritance via BaseClass for consistency,
one-to-many relationships between user and health data,
composition between posts and comments,
clear separation between patient and admin functionalities.

This model provides a solid foundation for backend implementation while keeping the business logic understandable for end users.

Directory structure

```
backend/
├── core/
│   ├── base_model.py
│   ├── exceptions.py
│   └── utils.py
├── users/
│   └── user.py
├── health/
│   ├── glycemia.py
│   ├── weight.py
│   ├── activity.py
│   ├── meal.py
│   ├── medication.py
│   └── mood.py
├── professionals/
│   ├── health_professional.py
│   └── note.py
└── community/
    ├── forum_post.py
    ├── comment.py
    └── article.py
```

FRONTEND ORGANIZATION

Page / Main Component: Auth

Sub-components:
- AuthLanding
- LoginForm
- RegisterForm
- PasswordResetForm
- ProfilePage

Role / Interaction:
- Authentication and account creation page.
- AuthLanding redirects to Login or Register.
- LoginForm manages login + UX options (“Remember me”, reset).
- RegisterForm also used for profile editing.
- Access to internal pages is protected via ProtectedRoute.

Page / Main Component: Dashboard

Sub-components:
- HeaderDashboard
- SidebarNav + NavItem
- NewsPreview
- DataCard (generic) with variants (GlycemiaCard, WeightCard, etc.)
- FormModals for each data entry (add via “+”)
- StatsCard
- Charts (glycemia, weight, activity, mood)
- DateSelector

Role / Interaction:
- Synthetic view for day/week/month.
- DataCard displays value + chart + “+” button.
- Forms appear in a modal when clicked.
- NewsPreview shows the 3 most popular articles (likes).
- SidebarNav serves as global navigation (profile, forum, calendar...).

Page / Main Component: Notes / Consultations

Sub-components:
- NotesPage
- NotesList
- NoteCard
- NoteDetail
- NoteForm

Role / Interaction:
- Management of personal notes linked to consultations.
- NotesList shows all notes sorted by date.
- NoteCard shows a summary, click opens NoteDetail.
- NoteForm for creating/editing/deleting a note.
- Notes are always associated with a health professional.

Page / Main Component: Calendar

Sub-components:
- CalendarView
- DayDataView
- DataFormModal

Role / Interaction:
- Access to history: view, add, edit, delete data for a specific day.
- CalendarView selects a date.
- DayDataView displays data for that day (mini-dashboard).
- DataFormModal manages CRUD for that day’s data.

Page / Main Component: Forum

Sub-components:
- ForumPage
- PostCard
- PostDetail
- PostForm
- CommentList
- CommentForm
- LikeButton

Role / Interaction:
- Community space.
- ForumPage lists posts as cards.
- PostCard shows title + excerpt + number of likes/comments.
- PostDetail shows full content + comments.
- PostForm creates/edits a topic.
- CommentList handles display, CommentForm adds new comment.
- LikeButton allows liking a post.

Page / Main Component: Health Professionals

Sub-components:
- ProfessionalsPage
- ProfessionalCard (simplified view)
- ProfessionalDetail (complete profile card)
- ProfessionalForm
- NotesPreview (latest note)
- NotesList (expand/collapse)
- NoteForm

Role / Interaction:
- Management of medical contacts.
- ProfessionalCard shows name + specialty + latest note.
- Click opens ProfessionalDetail (full info + edit).
- NotesPreview shows the most recent note.
- NotesList expands to show history of notes for a professional.
- ProfessionalForm adds/edits a professional.

Page / Main Component: Articles / News

Sub-components:
- ArticlesPage
- ArticleCard
- ArticleDetail
- ArticleForm (admin only)
- SearchBar
- LikeButton

Role / Interaction:
- Page of articles published by the admin.
- ArticlesPage displays articles as cards with search.
- ArticleDetail shows full content.
- ArticleForm enables CRUD (admin only).
- LikeButton updates popularity counter to highlight articles in the Dashboard.

Directory structure

```
src/
├── assets/
│
├── components/
│   ├── layout/
│   │   ├── HeaderDashboard.jsx
│   │   ├── SidebarNav.jsx
│   │   ├── NavItem.jsx
│   │   └── Footer.jsx
│   ├── ui/
│   │   ├── Button.jsx
│   │   ├── LikeButton.jsx
│   │   ├── SearchBar.jsx
│   │   └── Modal.jsx
│   ├── charts/
│   │   ├── ChartGlycemia.jsx
│   │   ├── ChartWeight.jsx
│   │   ├── ChartActivity.jsx
│   │   └── ChartMood.jsx
│   └── cards/
│       ├── DataCard.jsx
│       ├── StatsCard.jsx
│       ├── PostCard.jsx
│       ├── ArticleCard.jsx
│       └── ProfessionalCard.jsx
├── context/
│   └── AuthContext.jsx
├── hooks/
│   └── useAuth.js
├── pages/
│   ├── Auth/
│   │   ├── AuthLanding.jsx
│   │   ├── LoginForm.jsx
│   │   ├── RegisterForm.jsx
│   │   ├── PasswordResetForm.jsx
│   │   └── ProfilePage.jsx
│   ├── Dashboard/
│   │   ├── DashboardPage.jsx
│   │   ├── GlycemiaForm.jsx
│   │   ├── WeightForm.jsx
│   │   ├── ActivityForm.jsx
│   │   ├── MealForm.jsx
│   │   ├── MedicationForm.jsx
│   │   └── MoodForm.jsx
│   ├── Calendar/
│   │   ├── CalendarPage.jsx
│   │   ├── CalendarView.jsx
│   │   ├── DayDataView.jsx
│   │   └── DataFormModal.jsx
│   ├── Notes/
│   │   ├── NotesPage.jsx
│   │   ├── NotesList.jsx
│   │   ├── NoteDetail.jsx
│   │   └── NoteForm.jsx
│   ├── Professionals/
│   │   ├── ProfessionalsPage.jsx
│   │   ├── ProfessionalDetail.jsx
│   │   ├── ProfessionalForm.jsx
│   │   └── NotesList.jsx
│   ├── Forum/
│   │   ├── ForumPage.jsx
│   │   ├── PostDetail.jsx
│   │   ├── PostForm.jsx
│   │   ├── CommentList.jsx
│   │   └── CommentForm.jsx
│   └── Articles/
│       ├── ArticlesPage.jsx
│       ├── ArticleDetail.jsx
│       └── ArticleForm.jsx
├── routes/
│   ├── AppRouter.jsx
│   └── ProtectedRoute.jsx
├── services/
│   ├── api.js
│   ├── authService.js
│   ├── userService.js
│   ├── dataService.js
│   ├── noteService.js
│   ├── professionalService.js
│   ├── forumService.js
│   └── articleService.js
└── routes/
```

task
03

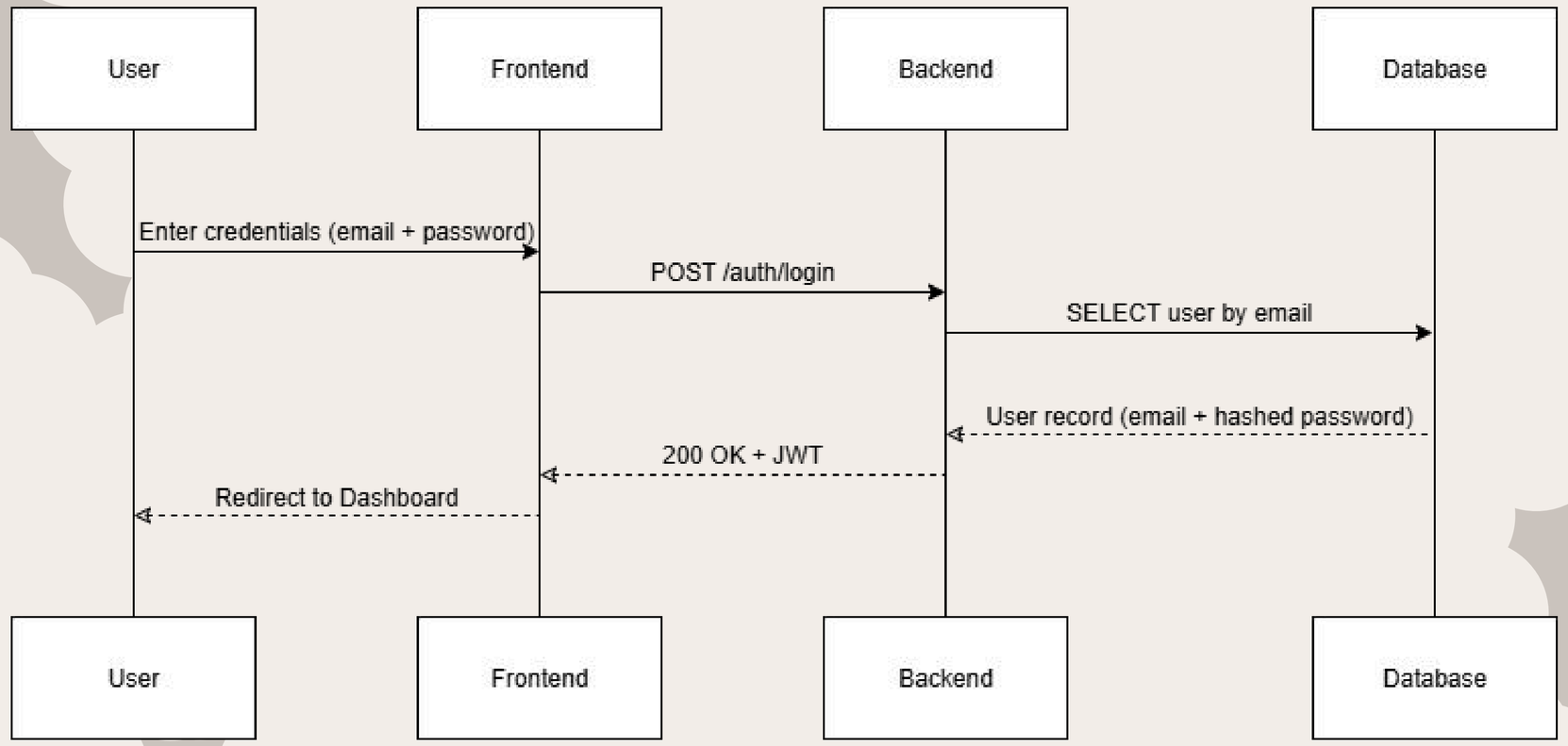
High-Level Sequence Diagrams

09.2025

CLAIRE CASTAN

HIGH-LEVEL SEQUENCE DIAGRAMS

LOGIN



STEP-BY-STEP WALKTHROUGH:

Step 1: User enters credentials

Description: The user types their email and password into the login form on the frontend interface.

Step 2: Frontend sends request

Description: Once the form is submitted, the frontend app sends an HTTP POST request to the backend endpoint /auth/login, containing the user's credentials in JSON format.

Step 3: Backend queries the database

Description: The backend receives the request, extracts the email, and queries the database (users table) to retrieve the corresponding record.

Step 4: Database responds

Description: The database returns the user record, which contains the email and the hashed password.

Step 5: Backend validates credentials

Description: The backend compares the provided password with the hashed password returned by the database. If they match, the backend generates a JWT (JSON Web Token) encoding the user's ID and role.

Step 6: Backend sends response

Description: The backend sends an HTTP 200 OK response to the frontend, along with the generated JWT. In case of error, it instead returns an error code (e.g., 401 Unauthorized).

Step 7: Frontend updates state

Description: The frontend stores the JWT (in AuthContext or localStorage) and updates the authentication state to mark the user as logged in.

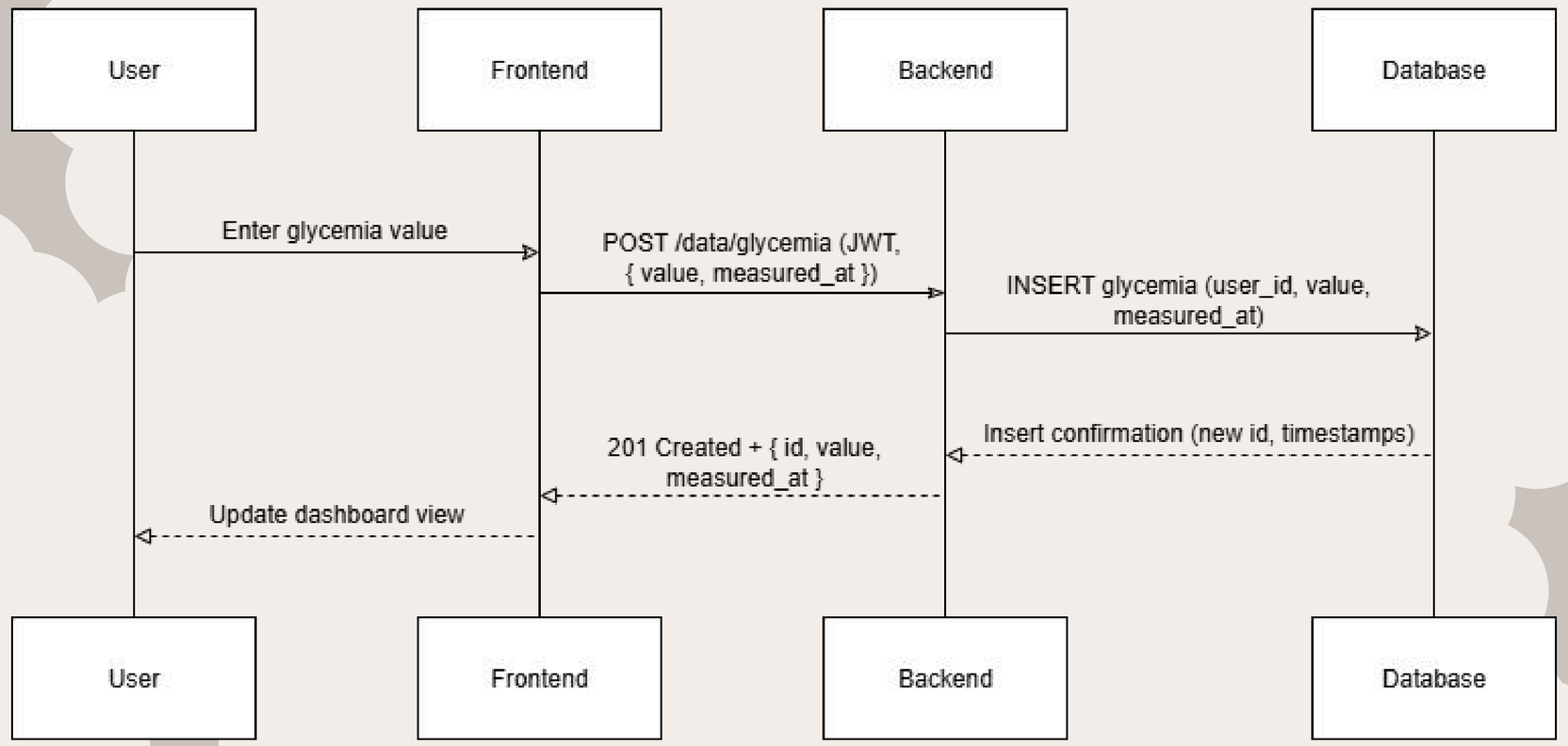
Step 8: User is redirected

Description: The authenticated user is redirected to the Dashboard page, where their personal data will be displayed.

HIGH-LEVEL SEQUENCE DIAGRAMS

09.2025

ADD GLYCEMIA



STEP-BY-STEP WALKTHROUGH:

Step 1: Enter value

Description: The user enters a new glycemia value (and date/time if applicable) in the Dashboard form/modal.

Step 2: Prepare request

Description: The frontend validates the format locally (number, boundaries), retrieves the JWT, and prepares the JSON body with value and measured_at.

Step 3: Send to API

Description: The frontend sends POST /data/glycemia with the header Authorization: Bearer <JWT> and the body { value, measured_at }.

Step 4: Auth & validation

Description: The backend verifies the JWT, checks that the user exists, and validates the fields (types, boundaries, date consistency).

Step 5: Write to database

Description: The backend inserts the record into the glycemia table (user_id, value, measured_at, timestamps).

Step 6: Database confirmation

Description: The database confirms insertion with the generated ID and timestamps.

Step 7: API response

Description: The backend responds with 201 Created and the stored resource { id, value, measured_at } (optionally with recalculated stats).

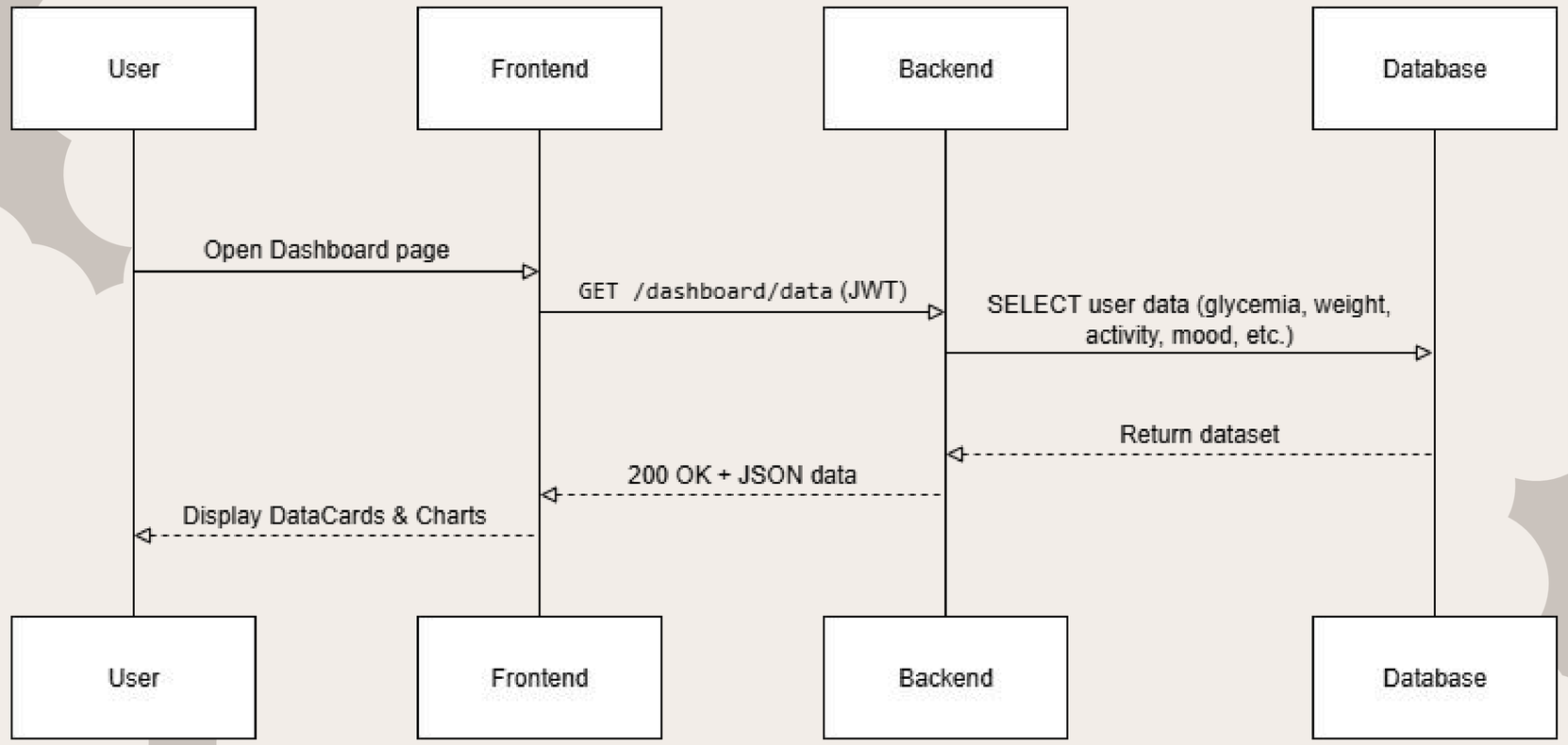
Step 8: UI update

Description: The frontend closes the modal, displays a success toast, and updates the card/chart (optimistic update or short Dashboard refetch to resync).

HIGH-LEVEL SEQUENCE DIAGRAMS

09.2025

DISPLAY DASHBOARD



STEP-BY-STEP WALKTHROUGH:

Step 1: Open page

Description: The user opens the Dashboard; by default, a time period is selected (e.g., 7 days) via the DateSelector.

Step 2: Data request

Description: The frontend reads the JWT and sends GET /dashboard/data?range=7d (or a from/to pair) to fetch the relevant datasets and statistics.

Step 3: API authentication

Description: The backend verifies the JWT, calculates the requested time window, and prepares database queries.

Step 4: Database read

Description: The backend queries the database for each relevant resource (glycemia, weight, activity, mood, etc.) filtered by the period.

Step 5: Return datasets

Description: The database returns the datasets.

Step 6: Aggregation & formatting

Description: The backend calculates statistics (min/average/max, possible trends) and structures a clear JSON response: datasets per resource + stats + period.

Step 7: API response

Description: The backend sends 200 OK with the aggregated JSON.

Step 8: UI rendering

Description: The frontend updates DataCards and Charts, manages “loading/empty/error” states, and synchronizes the DateSelector.

Conclusion

The three high-level sequence diagrams above illustrate the essential interactions of the Sweet application: authentication (Login), data creation (Add Glycemia), and data visualization (Display Dashboard). Together, they cover the two most critical flows of any health-tracking system: secure access and reliable data management.

- The Login diagram shows how credentials are validated and exchanged for a secure JWT, enabling protected access to user data.
- The Add Glycemia diagram illustrates how a new health record is submitted, validated, stored in the database, and immediately reflected in the dashboard UI.
- The Display Dashboard diagram shows how multiple datasets (glycemia, weight, activity, mood, etc.) and the most popular articles are aggregated and returned to the frontend in a single API response.

In all three scenarios, error handling is integrated consistently. Invalid credentials, missing or malformed input, expired tokens, or unexpected server/database errors trigger the appropriate HTTP code (4xx for client errors, 5xx for server errors). On the frontend, clear error messages are displayed and state updates are blocked, ensuring a reliable and transparent user experience.

This approach guarantees that the system is not only functional under ideal conditions but also resilient to failures, providing explicit feedback to users and preserving data integrity.

task
04

API Endpoints (Internal)

09.2025

AUTH :

Method	Endpoint	Input	Output	Notes
`POST`	`/auth/register`	`{ firstname, lastname, email, password, age, gender }`	`{ id, firstname, lastname, email, role }`	Create a new user account
`POST`	`/auth/login`	`{ email, password }`	`{ token (JWT), user: { id, firstname, role } }`	Authenticate user & return JWT
`POST`	`/auth/reset`	`{ email, new_password }`	`{ message: "Password reset successful" }`	Simple password reset
`GET`	`/auth/profile`	Header: `Authorization: Bearer <JWT>`	`{ id, firstname, lastname, email, age, gender }`	Get user profile
`PUT`	`/auth/profile`	`{ firstname?, lastname?, age?, gender?, email? }`	`{ updated user }`	Update user profile (must be logged in)

Note on authentication:

In production, JWT tokens are stored in secure httpOnly cookies, in order to protect sensitive medical data and minimize risks related to XSS attacks.
For easier testing and interactive documentation with Swagger, the API also supports JWT tokens sent in the Authorization header as a Bearer token.
This dual approach combines maximum security in production with practicality during testing.

DATA (DASHBOARD) : COVERS → GLYCEMIA, WEIGHT, ACTIVITY, MEAL, MEDICATION, MOOD.

Method	Endpoint	Input	Output	Notes
`POST`	`/data/glycemia`	`{ value, measured_at }`	`{ id, value, measured_at }`	Add glycemia record
`GET`	`/data/glycemia?range=7d`	Query param: range (`day`, `7d`, `30d`)	`{ records: [...], stats: { min, max, avg } }`	Fetch glycemia data + stats
`POST`	`/data/weight`	`{ value, measured_at }`	`{ id, value, measured_at }`	Add weight record
`GET`	`/data/weight?range=7d`	Query param: range	`{ records: [...], stats: { min, max, avg } }`	Fetch weight data
`POST`	`/data/activity`	`{ type, duration, measured_at }`	`{ id, type, duration, measured_at }`	Add physical activity
`GET`	`/data/activity?range=7d`	Query param: range	`{ records: [...] }`	Fetch activity data
`POST`	`/data/meal`	`{ description, measured_at }`	`{ id, description, measured_at }`	Add meal entry
`POST`	`/data/medication`	`{ name, dose, stock, taken_at }`	`{ id, name, dose, stock, taken_at }`	Track medication intake & stock
`POST`	`/data/mood`	`{ value, measured_at }`	`{ id, value, measured_at }`	Record mood (scale or text)

CALENDAR :

Method	Endpoint	Input	Output	Notes
`GET`	`/calendar/day/:date`	Param: date (`YYYY-MM-DD`)	`{ data: { glycemia[], weight[], ... } }`	Fetch all records for a specific day
`PUT`	`/calendar/day/:date`	`{ updates }`	`{ updated_data }`	Modify existing record
`DELETE`	`/calendar/day/:date/:id`	None	`{ message: "Deleted" }`	Delete a record by ID

NOTES (CONSULTATIONS) :

Method	Endpoint	Input	Output	Notes
`POST`	`/notes`	`{ professional_id, content, created_at }`	`{ id, professional_id, content, created_at }`	Add note linked to a professional
`GET`	`/notes/:professional_id`	None	`{ notes: [...] }`	Get all notes for a professional
`PUT`	`/notes/:id`	`{ content }`	`{ updated_note }`	Edit note
`DELETE`	`/notes/:id`	None	`{ message: "Note deleted" }`	Delete note

PROFESSIONALS :

Method	Endpoint	Input	Output	Notes
`POST`	`/professionals`	`{ professional_id, content, created_at }`	`{ id, name, specialty, phone, email }`	Add professional
`GET`	`/professionals`	None	`{ professionals: [...] }`	List all professionals
`GET`	`/professionals/:id`	None	`{ professional }`	Get details of one professional
`PUT`	`/professionals/:id`	`{ updates }`	`{ updated_professional }`	Update professional
`DELETE`	`/professionals/:id`	None	`{ message: "Professional deleted" }'	Delete professional

ARTICLES :

Method	Endpoint	Input	Output	Notes
`POST`	`/articles`	`{ title, content }`	`{ id, title, content, created_at, likes }`	Create new article (admin only)
`GET`	`/articles`	Query: search (optional), sort=popular	`{ articles: [...] }`	List articles, can be sorted by popularity (likes)
`GET`	`/articles/:id`	None	`{ article }`	Get one article
`PUT`	`/articles/:id`	`{ updates }`	`{ updated_article }`	Update article (admin only)
`DELETE`	`/articles/:id`	None	`{ message: "Article deleted" }`	Delete article (admin only)
`POST`	`/articles/:id/like`	None	`{ likes: count }`	Like an article

FORUM :

Method	Endpoint	Input	Output	Notes
`POST`	`/forum/posts`	`{ title, content }`	`{ id, title, content, created_at, likes }`	Create new post
`GET`	`/forum/posts`	Query: search (optional)	`{ posts: [...] }`	List all posts
`GET`	`/forum/posts/:id`	None	`{ post: {...}, comments: [...] }`	Get one post with comments
`PUT`	`/forum/posts/:id`	`{ updates }`	`{ updated_post }`	Edit post
`DELETE`	`/forum/posts/:id`	None	`{ message: "Post deleted" }`	Delete post
`POST`	`/forum/posts/:id/like`	None	`{ likes: count }`	Like a post
`POST`	`/forum/comments`	`{ post_id, content }`	`{ id, post_id, content, created_at }`	Add comment
`PUT`	`/forum/comments/:id`	`{ content }`	`{ updated_comment }`	Edit comment
`DELETE`	`/forum/comments/:id`	None	`{ message: "Comment deleted" }`	Delete comment

Conclusion

The internal API of the Sweet application was designed following REST principles, ensuring clarity, modularity, and scalability.
Each module — Authentication, Data, Calendar, Notes, Professionals, Forum, and Articles — is represented by a coherent set of endpoints covering the required CRUD operations.

- The Authentication API secures access with JWT tokens, supporting signup, login, password reset, and profile management.
- The Data and Calendar APIs provide reliable mechanisms for recording and retrieving health data (glycemia, weight, activity, meals, medication, mood), along with aggregated statistics over different periods.
- The Notes and Professionals APIs allow users to keep structured records of medical consultations, directly linking notes to health professionals.
- The Forum and Articles APIs support the community and educational features of the app, with endpoints to create, read, update, delete, and like posts or articles.

Error handling is standardized: client-side issues return 4xx codes, server-side errors return 5xx codes, and descriptive error messages are always included in the JSON response.

By centralizing all interactions through these well-defined endpoints, the Sweet system ensures a robust and extensible architecture, where the frontend can reliably consume data while enforcing authentication and authorization rules.

task
05

SCM & QA Stratégies

09.2025

CLAIRE CASTAN

SCM (SOURCE CONTROL MANAGEMENT)

1.Branching strategy

One branch per feature (feature branch).

Once the feature is finalized and tested, it is merged into the main branch.

This workflow isolates ongoing developments, keeps main stable, and makes the repository more understandable for external reviewers.

2.Commit convention (GitMoji)

Use of the GitMoji convention, which is highly visual. Example: :emoji: (folder/file.xx): commit_message.

Each commit starts with an emoji that visually describes the type of change:

✨ → new feature

♻️ → code refactor

🔨 → code fix

💄 → UI/style improvement

🐛 → bug fix

🧪 → test

...

3.Pull requests and merges

Since the project is developed by a single contributor, there is no formal Pull Request process.

Feature branches are merged directly into main once the functionality has been validated.

QUALITY ASSURANCE (QA)

Quality assurance is integrated throughout development to ensure reliability, maintainability, and user trust.

1.Centralized error handling

All error messages from the frontend, backend, and database are centralized into a log file.

This makes debugging easier, helps quickly identify recurring issues, and provides a single source for anomaly analysis.

2.Automated tests

Backend: unit tests with Pytest to validate models, methods, and API endpoints.

Frontend: unit tests with Jest and React Testing Library to verify rendering and component behavior.

3.API testing and documentation

The backend API is documented and tested with Swagger, which provides an interactive interface to explore endpoints.

Swagger replaces manual tests with Postman and serves both as a testing tool and as living documentation.

4.Code quality and linting

Frontend: ESLint + Prettier to comply with JavaScript/React standards.

Backend: Flake8 to comply with Python's PEP8 convention.

These tools ensure code that is clean, consistent, and professional.

5.Manual validation

Critical user flows (login, adding a data entry, displaying the dashboard) are manually tested after development.

Conclusion

The SCM and QA strategies implemented in Sweet ensure a professional workflow, even for a solo project.

- On the SCM side, using feature branches and GitMoji commits makes the Git history readable and traceable.

- On the QA side, the combination of centralized logging, automated tests, Swagger, linting, and manual validation guarantees that the project is both functional and robust.

These practices ensure that Sweet remains a maintainable and scalable project over the long term.

task
06

Technical Justifications

09.2025

TECHNICAL JUSTIFICATIONS

09.2025

The choice of technologies and the overall design of the Sweet application is based on a balance between simplicity, scalability, security, and an adapted working methodology.

Frontend

React was chosen because it follows a logic of reusable components, perfectly suited for an individual project. This approach helps organize code more effectively, makes testing easier, and reduces duplication.

This choice also aligns with the development team's way of working, which prioritizes decomposing features, clear organization, and reusability of components.

The use of React Router is essential to handle navigation between different pages (Dashboard, Forum, Articles, Calendar, etc.) in a smooth and standardized way.

Finally, React is also a strategic choice: it enables a natural evolution toward React Native, allowing the web application to be transformed into a mobile app by reusing part of the existing logic and services.

Backend

Flask was selected for its lightness, simplicity, and ability to structure a clear RESTful API.

Its modular architecture with Blueprints allows each domain (auth, data, forum, articles, health professionals) to be isolated, improving maintainability and scalability.

Using SQLAlchemy with Flask-Migrate simplifies database management while preventing SQL injection risks.

Database

MySQL was chosen for its stability, compatibility with SQLAlchemy, and strong handling of complex relationships between users, health data, professionals, and community features.

The utf8mb4 encoding ensures compatibility with special characters and emojis.

The structure follows a clear, normalized model that guarantees confidentiality (each user only sees their own data).

Security

Authentication relies on JWT tokens stored in secure httpOnly cookies, greatly limiting risks of XSS attacks.

Passwords are secured with bcrypt, and a minimum complexity policy is enforced (length, number, letter).

Input validation is strengthened through the use of Regex on sensitive fields (emails, passwords, phone numbers), along with boundary checks for numeric data (glycemia, weight, activity).

Data transport is secured with HTTPS, using secure cookies and a strict CORS policy.

Organization and best practices

An abstract BaseClass centralizes CRUD methods and common attributes, ensuring consistency across backend entities.

Chart.js is used for data visualization, making charts clearer and more educational—essential in a medical tracking context.

Eco-design considerations are integrated through lightweight technical choices (modular SASS, simple animations, request size limitations).

GENERAL CONCLUSION

The technical documentation of Part 3 of Sweet demonstrates a design that is complete, coherent, and aligned with the project's objectives.

- The user stories and mockups ensure that user experience remains a priority, covering both essential MVP needs and future evolution paths.
- The three-layer system architecture (frontend, backend, database) guarantees a clear separation of responsibilities, better maintainability, and natural scalability.
- The software design (class diagrams, database, frontend and backend structures) highlights rigorous modeling, adhering to best practices in normalization, security, and eco-design.
- The sequence diagrams describe the application's critical scenarios: authentication, adding health data, and visualizing data, ensuring transparency in flows and error handling.
- The API endpoints cover all functionalities (authentication, data, calendar, notes, health professionals, forum, articles), following RESTful principles and standardized error handling.
- The SCM and QA strategies lay the foundations for a professional workflow, integrating Git with commit conventions, centralized logging, automated tests, API documentation via Swagger, linting, and manual testing of critical flows.
- Finally, the technical justifications clearly explain the choices made (React + React Router, Flask, MySQL, JWT, Regex, GitMoji, etc.), showing that they address both the constraints of an MVP and the ambitions of an evolving project, including a future transition to React Native.

By bringing all these elements together, Sweet positions itself as a secure, robust, and scalable application, capable of addressing the challenges of digital health tracking.

This Part 3 provides a solid foundation to guide the development of the technical implementation (Part 4), ensuring that every decision is based on thoughtful and well-justified design.